

Jason Zhao

CSc 59866

Kurt Brown

Candidate Problem Background: Role-Awareness Communication and Coordination System

Organizations and software teams often need people from different groups and roles to coordinate work. A task may require information from another department, assistance from a technical team, approval from a specific role, or contributions from several people. These situations can become more difficult as organizations grow, responsibilities change, and teams become distributed. Currently, coordination is commonly handled through direct messages, group conversations, meetings, email, documentation, and other communication tools. These approaches allow people to exchange information, but they often require users to know already who is responsible for a particular task or where the necessary expertise is located. Research on large-scale software development has found that teams need to coordinate with other teams, experts, and supporting roles, and that locating appropriate expertise and managing dependencies can create coordination challenges (Sablis et al., 2020). The problem I want to investigate is how communication and coordination can be organized when responsibilities belong to roles or groups rather than only to specific individuals. This could become particularly relevant when multiple teams are involved, responsibilities change, or a request needs to be tracked from creation through completion.

The proposed project is a **role-aware communication and coordination system**. Instead of treating communication primarily as messages between individual users, the system would make organizational roles and groups part of the communication structure.

Each user would have information such as their name, organizational role, and group. Users could communicate with three types of recipients:

- **Individual:** a specific user.
- **Role:** the person or person currently assigned to a particular role.
- **Group:** everyone belonging to an organizational group.

For example, a Product Manager could send a request to the Software Engineering role without needing to know which engineer currently handles that responsibility. The system would associate the request with the appropriate users and display the sender's name, role, and group to provide organizational context. Lyu et al. (2022) discuss transactive memory systems, where team members divide knowledge among people with different areas of expertise and rely on knowledge about who possesses particular expertise. This provides a basis for investigating whether explicitly representing roles and responsibilities can help users identify where relevant knowledge or responsibility is located. The system would also support structured requests rather than treating every interaction as an ordinary message. A request could contain a description, recipient, deadline, current status, acknowledgment, and completion of information. For example:

Created → Acknowledged → In Progress → Completed

A user could therefore see whether a request has been received, whether someone has taken responsibility, and whether the work has been completed. The system would also need to handle changing organizational conditions. If someone changes roles, the system will update the users associated with that role. If multiple people respond to a group request, the system will need to determine how those responses affect the request status. If nobody acknowledges a request, the system could record it as unacknowledged and generate a reminder. These features are proposed design ideas that would need to be implemented and tested rather than assumed to work. Research on distributed software development identifies communication and coordination as challenges associated with distributed development. However, Bird et al. (2009) found in their study of Windows Vista that distributed development did not automatically result in worse software quality. This suggests that distribution itself should not be treated as proof of a problem. Instead, the project would investigate specific coordination mechanisms that could support teams working across organizational boundaries.

Approach	How it addresses the problem	Strengths	Limitations
Direct Messages	Users contact a specific person through messages, email, meetings, or similar tools.	Simple, flexible, and familiar.	Users must know who is responsible; responsibility can become unclear when roles change.
Group-based Messages	Users communicate through team channels or group conversations.	Allows information to reach multiple people.	A group message does not necessarily identify who is responsible

			for responding or completing the request.
Proposed role-awareness system	Communication can target individuals, roles, or groups, while requesting to track responsibility and status.	Makes organizational context and request ownership explicitly.	Requires additional logic for role changes, concurrent responses, and synchronization. Effectiveness remains untested.

The goal is not to demonstrate that existing approaches are ineffective. Instead, the project asks whether roles and responsibilities can be treated as first-class parts of communication.

Project option: This currently appears closer to **Option 1**, because the project would involve designing and implementing a software system around a defined coordination problem.

Type of design: It would primarily involve **original design**, while incorporating established ideas from messaging, task management, organizational databases, and distributed systems.

Distributed systems relevance: The system would involve multiple clients communicating with backend services over a network. Shared states would include users, roles, groups, messages, requests, acknowledgments, and statuses. Concurrent activity could occur when multiple users respond to the same request or when role information changes while requests are being processed. A possible architecture could contain a client application, authentication/user service, organizational-role service, messaging service, and request service. This creates opportunities to investigate network communication, shared state, synchronization, consistency, concurrent updates, service failures, reliability, and scalability.

Questions	Possible Solutions/ Investigation
Does role-based communication make it easier to route requests?	Compare role-based and individual-based workflows using a prototype.
How should multiple people responding to one request affect ownership and completion?	Implement and test different acknowledgment and ownership rules.

Can shared request and role information remain consistent during concurrent updates?	Conduct distributed systems for tests involving simultaneous operations and failures.
--	---

The main early risk is scope. A full communication platform could require authentication, messaging, notifications, file sharing, search, task management, and many other features. Attempting to reproduce all of these would make the project too large. The initial project would therefore focus on role/group communication and structured request tracking. Another risk is evaluation, since demonstrating reduced coordination effort may require realistic users or controlled experiments.

The Bird et al. study shows that distributed development should not automatically be treated as a source of poor outcomes. This encourages the project to focus on specific coordination mechanisms rather than distribution itself (Bird et al., 2009).

The Lyu et al. study's discussion of transactive memory supports the idea of representing expertise and responsibilities within the system. It suggests that knowing who possesses particular knowledge is relevant to team coordination (Lyu et al., 2022).

The Sablis et al. study most directly motivates the candidate because it examines coordination between teams, experts, and supporting roles in large-scale software projects. Its discussion of locating expertise and coordinating interdependent work provides a basis for investigating role-aware communication (Sablis et al., 2020).

Overall, this project would investigate whether making roles, groups, responsibilities, and request states explicit can provide a useful way to coordinate organizational work while creating meaningful distributed-systems challenges involving shared state, concurrency, synchronization, and reliability.

References

Bird, C., Nagappan, N., Devanbu, P., Gall, H., & Murphy, B. (2009). Does distributed development affect software quality? An empirical case study of Windows Vista. *Proceedings of the 31st International Conference on Software Engineering*, 518–528. IEEE Computer Society. [Microsoft Research](#)

Lyu, B., Liao, X., & Yang, Y. (2022). Relationships between temporal leadership, transactive memory systems and team innovation performance. *Psychology Research and Behavior Management*, 15, 2543–2559. [PMC full text](#)

Sablis, A., Smite, D., & Moe, N. B. (2020). Team-external coordination in large-scale software development projects. *Journal of Software: Evolution and Process*, 33(3), e2297.
[Wiley full text](#)